

Primitive vs. General Recursion

Learning Outcomes

by the **End of the Lecture, Students** that **Complete** the **Recommended Exercises** should be **Able** to:

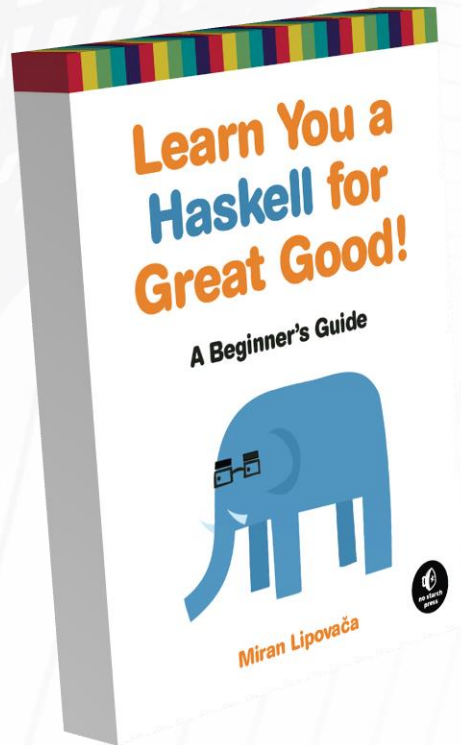
Define Primitive Recursion

Define General Recursion

Describe Insertion Sort as a Recursive Process

Create a Recursive Implementation of Insertion Sort

Reading and Suggested Exercises



Read the following **Chapter(s)**:

Chapter 5:
"Quick, sort!"
"Thinking recursively"

General Recursion

Primitive Recursion for Lists has a **Base Case** at `[]` and a **Recursive Case** at `h : t` such that the **Argument for Each Recursive Call** is `t`

General Recursion does **Not** share these **Constraints**

it may have **More Than One Base Case**

it may have **Different or Multiple Recursive Arguments**

the **Arguments** may **Not** be **Strictly Smaller**

Insertion Sort is a **Sorting Algorithm** that **Starts** with an **Unsorted List** and a **New, Empty List** into which the **Elements** of the **Unsorted List** are **Inserted**

the **Insertion Sort Operator** considers **Each Unsorted Element** in turn before **Moving** it to its **Correct Location** in the **New List**

**How would you Write a Recursive
Implementation of the Insertion Sort Operator?**

General Recursion

```
insertOp :: Int -> [Int] -> [Int]
```

the **First Argument** is the **Element** to be **Inserted**
and the **Second Argument** is the **List** into which
the **Element** must be **Inserted**

Which Argument is the Recursive Argument?

What will be the Base Case?

General Recursion

```
insertOp :: Int -> [Int] -> [Int]  
  insertOp x [] = something  
  insertOp x (h:t) = something
```

When the Head is Compared with the Element to be Inserted, What are the Possible Responses?

General Recursion

once the **Insertion Sort Operator** is written,
How do you **Write** a **Recursive Function** for
Performing Insertion Sort?

```
insertSort :: [Int] -> [Int]
```

What is the Base Case?
What is the Recursive Case?

General Recursion

although **General Recursion** is **Less Constrained** than **Primitive Recursion**, **Primitive Recursion** has **Significant Advantages** (essentially **Guarantees**):

Primitive Recursion Always Terminates

All Legitimate Arguments are Handled

How would you Write a Function that "Zips" Two Lists into One (of the Lesser Length) of Tuples

```
zip :: [a] -> [b] -> [(a, b)]
```

```
zip [1, 3, 25] ['a', 'c', 'y', 'z'] =  
  [(1, 'a'), (3, 'c'), (25, 'y')]
```

Why won't this Function be Primitive Recursion?